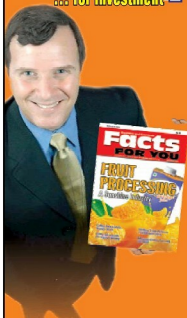


Grab
your copy
today

Look Before You Leap

...for investment



A business and economic affairs magazine that influences corporate decisions on the line of **Where should we put our money next?**

Launched in 1980, Facts For You updates India's top decision-makers through its regular assortment of market surveys and other important industrial information.

For subscription & advertisements, contact:



EFY Enterprises Pvt Ltd

D-87/1, Okhla Industrial Area, Phase-I,
New Delhi - 110020

Phone: 011-26810601/02/03

Email: info@efyindia.com

Website: www.efymag.com

```
not modify I */
Result = I * 10;
Return Result;
}
Int Bar()
{
    Int I = 20;
    Int Res = Foo(I);
    Return res;
}
```

In the above code snippet, it is not possible to perform classical constant propagation in the function 'foo' since the variable 'I' is the formal parameter for the function 'foo', and without inter-procedural analysis, the compiler cannot determine whether the formal parameter 'I' in 'foo' is a constant. However, when inter-procedural constant propagation is enabled as part of the inter-procedural analysis (IPA) phase of the compiler, the compiler creates a summary for each call site. It recognises that in all calls involving the call to 'foo' in the above code snippet, the function 'foo' is passed the argument value of 20. Hence, it can propagate this constant value into the called function 'foo', and then perform the classical constant propagation within the function 'foo'. Some of the other popular inter-procedural transformations are inter-procedural range propagation, procedure boundary elimination, function cloning, partial inlining and structure layout transformations. We will discuss a few of them in next month's column.

My 'must-read book' for this month

This month's recommendation comes from one of our readers, K Sunil, who

suggests we read a classic to learn more about compiler optimisations. The book is the *Compiler Handbook* edited by Prof Srikanth and Prof Priti Shankar. The book is a compilation of articles on different compiler optimisations, covering both theoretical and practical aspects. Thank you, Sunil, for the recommendation. While this is a must-read book for anyone wanting to become a compiler engineer, this is probably not the best choice for those just learning compilers as their first course. For a beginner's course in compiler optimisations, I would suggest readers consult the classic 'dragon book', namely, *Compiler Design* by Aho, Sethi and Ullman. But if you are interested in delving into many of the state-of-the-art compiler optimisations, then Sunil's recommendation of the *Compiler Handbook* is a must-have on your bookshelf. The chapters on memory hierarchy optimisations and register allocation make for interesting reading. The book contains a number of references to many of the classical compiler papers.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in this column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming, and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software and works at Hewlett-Packard India. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.